arXiv:2505.10978v1 [cs.LG] 16 May 2025

Group-in-Group Policy Optimization for LLM Agent Training

Lang Feng[1] **Zhenghai Xue**[1] **Tingcong Liu**[1] **Bo An**[1,2]
[1]Nanyang Technological University, Singapore
[2]Skywork AI, Singapore

Abstract

Recent advances in group-based reinforcement learning (RL) have driven frontier large language models (LLMs) in single-turn tasks like mathematical reasoning. However, their scalability to long-horizon LLM agent training remains limited. Unlike static tasks, agent-environment interactions unfold over many steps and often yield sparse or delayed rewards, making credit assignment across individual steps significantly more challenging. In this work, we propose *Group-in-Group Policy Optimization (GiGPO)*, a novel RL algorithm that achieves fine-grained credit assignment for LLM agents while preserving the appealing properties of group-based RL: critic-free, low memory, and stable convergence. GiGPO introduces a two-level structure for estimating relative advantage: (i) At the *episode-level*, GiGPO computes macro relative advantages based on groups of complete trajectories; (ii) At the *step-level*, GiGPO introduces an *anchor state grouping* mechanism that retroactively constructs step-level groups by identifying repeated environment states across trajectories. Actions stemming from the same state are grouped together, enabling micro relative advantage estimation. This hierarchical structure effectively captures both global trajectory quality and local step effectiveness without relying on auxiliary models or additional rollouts. We evaluate GiGPO on two challenging agent benchmarks, ALFWorld and WebShop, using Qwen2.5-1.5B-Instruct and Qwen2.5-7B-Instruct. Crucially, GiGPO delivers fine-grained per-step credit signals and achieves performance gains of > 12% on ALFWorld and > 9% on WebShop over the GRPO baseline: all while maintaining the same GPU memory overhead, identical LLM rollout, and incurring little to no additional time cost.

1 Introduction

Large Language Models (LLMs) [1, 2, 3, 4] have leapt from static question-answer systems to versatile *agents* that perceive, reason, and act in open-ended environments. For instance, they now power embodied assistants that navigate simulated homes [5, 6], mobile and web navigators that plan multi-step browsing sessions [7, 8, 9], and autonomous explorers in interactive games [10, 11]. In these settings, LLM agents need to perceive, reason, and act in multi-turn loops, which requires not only language understanding but also long-horizon planning and decision-making.

Reinforcement learning (RL) [12] has become a crucial recipe for post-training LLMs, leading to frontier models like OpenAI o1 [13] and DeepSeek R1 [14]. In particular, group-based RL algorithms such as RLOO [15, 16] and GRPO [17] have proven especially effective in large-scale training. These methods replace value-function estimation with simple yet powerful relative advantage estimation within groups of rollouts. This group-based advantage computation enjoys favorable properties such as low memory overhead, critic-free optimization, and scalability to large models. However, their

Code: https://github.com/langfengQ/verl-agent

Preprint. Under review.

successes have so far been largely limited to single-turn tasks such as math problem solving [18, 19] and code generation [20], where reward arrives immediately and credit assignment is straightforward.

In contrast, LLM agents operating in external environments face fundamentally different learning landscapes. Their behavior unfolds over long episodes with tens of decision steps and tens of thousands of tokens (e.g., an ALFWorld [5] episode may include up to 50 steps and over 20k tokens). Rewards are typically sparse (sometimes arriving only at the end of an episode), and the impact of any individual action may only manifest much later in the trajectory. This substantially complicates the credit assignment for individual steps and increases the challenge of policy optimization. Naively applying existing group-based RL algorithms in such settings collapses step-level distinctions, undermining their effectiveness. Hence, these limitations raise a core question:

Can we preserve the critic-free, low-memory, and stable convergence properties of group-based RL while introducing fine-grained credit assignment for long-horizon LLM agents?

To address this, we introduce *Group-in-Group Policy Optimization* (GiGPO), a new group-based RL algorithm that nests two-dimensional notions of credit assignment, better-suited for the multi-step optimization of LLM agents. (i) At the *episode level*, GiGPO samples a group of complete trajectories under identical task and initial-state conditions, and computes macro relative advantages based on total returns like vanilla GRPO [17]. This captures the overall effectiveness of each trajectory and reflects the completeness of task execution. (ii) At the *step level*, GiGPO introduces a novel *anchor state grouping* mechanism for fine-grained relative advantages estimation. Specifically, it retroactively identifies the repeated environment states, termed as *anchor states*, across trajectories and uses them as anchors to construct step-level groups that allow for localized credit assignment.

The key insight behind GiGPO is that, under identical tasks and initial environment conditions, many trajectories within the group encounter the same states *multiple times* due to ineffective actions or loops, such as revisiting the same webpage, room, or game scene. These shared states provide a natural basis for step-level group construction and computing more granular advantage estimates. GiGPO uses these step-level groups to assign localized credit to actions based on their relative performance at a common state, enabling more precise optimization while avoiding the cost explosion of per-step extra rollouts. As such, GiGPO remains fully critic-free and requires no auxiliary value models while dramatically introducing finer credit signals for training LLM agents.

We evaluate GiGPO on two long-horizon agent benchmarks: ALFWorld [5], which tests embodied task planning in simulated household environments, and WebShop [21], which simulates complex, goal-driven web interactions. Our experiments with Qwen2.5-1.5B-Instruct and Qwen2.5-7B-Instruct [3] show that GiGPO consistently outperforms prompt-based agents, actor-critic baselines, and prior group-based RL methods. In particular, GiGPO injects fine-grained, step-level credit signals that sharpen policy learning of agents over long horizons and achieves performance improvement of $> 12\%$ on ALFWorld and $> 9\%$ on WebShop over GRPO. These gains come without compromising the core strengths of group-based RL (only $< 0.002\%$ time cost), making GiGPO a versatile and high-utility algorithm for the long-horizon optimization of LLM agents.

2 Related Work

LLMs as decision-making agents. The use of large language models (LLMs) as autonomous agents has expanded rapidly across domains such as program generation [22], smart device operation [23, 24, 25, 26], interactive gameplay [10], and robot behavior control [27]. Early works typically relied on leveraging pre-trained, frozen models through carefully designed prompting methods (like ReAct [28] and Reflexion [29]), enhanced memory and retrieval systems [11, 30], and integration with external tools [31, 32, 33]. More recent research has shifted toward adapting model parameters with supervised fine-tuning (SFT) [23] or RL [12], enabling agents to learn directly from environment interaction rather than static prompts or handcrafted workflows, which we introduce below.

Reinforcement learning for LLM agents. RL has played a pivotal role in enabling LLM agents to operate in dynamic, open-ended environments. Early work applied classical RL algorithms such as DQN [34] to train LLM agents in text-based games [35] and later research [36, 37, 38, 39, 40] started to employ value-based methods, such PPO [41] and AWR [42], in more diverse and interactive agent scenarios including Android device control [43], embodied ALFWorld [5], and card games [44]. More recent approaches have extended RL training to complex web-based and application-centered tasks.

For instance, ArCHer [45] and AgentQ [46] target the WebShop benchmark [21], but require intricate designs and computation overhead such as additional value networks or Monte Carlo Tree Search (MCTS) [47]. Going further, LOOP [48] introduces a hybrid method combining REINFORCE leave-one-out (RLOO) [15, 16] with PPO-style updates, achieving state-of-the-art results in AppWorld [49]. RAGEN [50] introduces a trajectory-level GRPO that concatenates all states, intermediate reasoning, and actions into a unified episode-level response. However, it faces scalability challenges in long-horizon tasks (e.g., in ALFWorld, which involves up to 50 steps).

Reinforcement learning for large language models. An early and influential application of RL in LLMs is the Reinforcement Learning from Human Feedback (RLHF) [51, 52, 53, 54], which focuses on aligning LLMs to human preferences. Most recent works have explored using RL to enhance the capabilities of reasoning and logical deduction in LLMs [55, 14]. In particular, group-based RL algorithms have emerged as promising alternatives to traditional methods like PPO [41]. These methods, such as RLOO [15, 16], GRPO [17], Dr. GRPO [18], DAPO [19], and CPPO [56], avoid introducing extra value functions by leveraging a group of samples from the same query and estimate the advantages accordingly. This enables the large-scale RL training and has shown strong results in tasks such as mathematical reasoning [14], search [57, 58], and tool use [59, 60]. Our work is closely related to this line of research, with a focus on training *LLM agents*. We aim to retain the benefits of group-based RL, such as critic-free learning and efficiency, while introducing finer-grained credit assignment. Moreover, the hierarchical core of GiGPO is orthogonal to existing group-based RL approaches, making it fully compatible and capable of incorporating them to enhance performance.

3 Preliminaries

Problem setup. We consider a general setting in which an LLM agent interacts with an environment to complete multi-step tasks based on a task description $x \in p(X)$. At each discrete time step $t = 1, 2, \ldots, T$, the agent observes a state $\boldsymbol{s}_t \in \mathcal{S}$ and generates a textual action $\boldsymbol{a}_t \in \mathcal{V}^n$, where $\mathcal{V}$ denotes the token vocabulary and n is the maximum generation length. The environment then returns a scalar reward $r_t \in \mathbb{R}$ and the next state $\boldsymbol{s}_{t+1}$. A full episode consists of a trajectory $\boldsymbol{\tau} = \{(\boldsymbol{s}_1, \boldsymbol{a}_1, r_1), (\boldsymbol{s}_2, \boldsymbol{a}_2, r_2), \ldots, (\boldsymbol{s}_T, \boldsymbol{a}_T, r_T)\}$. The agent's behavior is governed by an LLM policy $\pi_\theta(\boldsymbol{a}_t | \boldsymbol{s}_t, x)$, parameterized by θ, which defines a distribution over outputs conditioned on the current state $\boldsymbol{s}_t$ and the task prompt x. In many realistic scenarios, the environment may provide sparse or delayed rewards (e.g., success and failure signals at the end of an episode) or weak feedback signals for intermediate steps. As the agent generates T consecutive textual actions $(\boldsymbol{a}_1, \ldots, \boldsymbol{a}_T)$, each potentially spanning thousands of tokens, it becomes particularly challenging to assign credit to individual tokens over the course of an episode.

Group-based RL. Recent RL works converge on a simple recipe for training LLMs: for a given task description x, the LLM samples a group of N candidate trajectories $\{\boldsymbol{\tau}_1, \boldsymbol{\tau}_2, \ldots, \boldsymbol{\tau}_N\}$, each corresponding to one full episode rollout under $\pi_{\theta_{\text{old}}}$. Each trajectory $\boldsymbol{\tau}_i$ receives a scalar reward $R(\boldsymbol{\tau}_i)$ reflecting the overall quality or success of the generated outcome. Instead of estimating advantages using separate value functions like PPO [41], group-based RL methods compute advantages purely based on the statistics internal to the sampled group:

$$A(\boldsymbol{\tau}_i) = \texttt{GroupComputation}(\{R(\boldsymbol{\tau}_i)\}_{i=1}^N). \tag{1}$$

For example, in GRPO [17], the advantage of each trajectory is estimated by normalizing its reward with respect to the group's mean and standard deviation. This design is highly memory-efficient and can scale effectively to the large batch sizes and model sizes typical in modern LLM training, making it a practical and scalable choice for large-scale RL training.

4 Training LLM Agents with GiGPO

While group-based RL algorithms [17, 14] have proven highly effective for training LLMs in single-turn tasks, their extension to multi-step agent settings faces critical challenges in credit assignment. Figure 1 illustrates this gap. Vanilla GRPO (left) treats each trajectory as a whole and computes a single relative advantage for the entire episode, which fails to provide actionable feedback for individual steps. A natural remedy is to roll out multiple single-step actions for each state $\boldsymbol{s}_t$ via $\pi_{\theta_{\text{old}}}$ as shown in Figure 1 (middle). However, this approach quickly becomes impractical due to